

Travel Tracker Learning Review

A summary of the debugging conversation, skill growth, strengths, and areas to improve while building the multi-user PostgreSQL Travel Tracker.

Overall assessment: You are moving from copying patterns into understanding how routes, forms, database rows, and EJS rendering connect. The main gap is not effort; it is consistently tracking data shape and request flow from the browser to Express to PostgreSQL and back to EJS.

Context

This review covers the part of our conversation focused on the Udemy Family Travel Tracker project. The goal was to add multi-user functionality to an Express, EJS, and PostgreSQL app, while keeping the guidance step-by-step and avoiding full code answers unless needed.

The project required the app to show different visited countries for different users, let the active user add countries, switch between users, and create a new user with a selected map color.

What We Worked Through

Area	What happened in the conversation
Map display	You first struggled with making visited countries light up on the SVG map. The issue came from passing full row objects instead of clean country-code strings into EJS and then into the browser script.
Home route	You moved from using one global country list to querying countries by the active user's id. The important shift was understanding that the home route should fetch users, find the current user, fetch only that user's visited countries, then render those values.
Add country route	You rebuilt the /add route from scratch. You learned that the country name typed into the form must be looked up in the countries table, then the resulting country_code must be inserted into visited_countries together with the current user id.
Switch user route	You separated the responsibility of /user: if the add-family-member button is clicked, show the new-user form; otherwise, update currentId from the selected user value and redirect home.
New user route	You used INSERT ... RETURNING * to create a user and immediately make that inserted user the active user. The key correction was remembering that insertion.rows is an array, so the inserted user's id lives at insertion.rows[0].id.

Where You Did Well

- You stopped blindly following the starter code and began rebuilding the routes in your own words. That is a major sign of real learning.
- You were willing to isolate one route at a time instead of trying to fix the whole app at once.
- You used console output and pgAdmin screenshots to compare what your code thought was happening against what was actually in the database.
- You understood that database query results come back as rows and started using `response.rows[0].country_code` correctly when only one result was needed.
- You picked up the purpose of SQL parameters: values like country code and current user id must be passed through the parameter array, not manually mixed into the query string.
- You handled the `/user` route well once the route responsibility became clear: selected existing user updates `currentId`; add-user button opens the new-user page.
- You kept asking for explanation instead of just asking for final code. That is slower in the moment, but better for long-term skill.

Where You Need To Improve

- Data shape discipline: pause and identify whether a value is an array, an object, a single row, or a plain string before using `map`, `find`, dot notation, or indexes.
- Form-to-route flow: always check the form action, method, and name attributes first. Those decide what route receives the request and what appears inside `req.body`.
- Route responsibility: each route should have a clear job. For example, `/add` adds a country, `/user` switches or opens user creation, `/new` creates a user, and `/` renders the current page state.
- Database boundary thinking: PostgreSQL does not know JavaScript variables unless you pass them in as query parameters. This is the reason `[codes, currentId]` mattered.
- Avoid reaching for `.map()` by habit. Use `map` when converting many rows. Use `rows[0]` when you only expect one selected row.
- Error handling: some catch blocks only log errors. In Express, a route should still respond or redirect after an error so the browser does not hang.
- Naming: names like `codes` can be confusing when the value is one country code. Clear names reduce mental load when the project grows.
- Edge cases: the current app still needs protection for duplicate country inserts, missing users, invalid user ids, and cases where no color is selected.

Technical Concepts To Practice Next

- Trace one request from the button click to the route, through the SQL query, and back into EJS.
- Write down the shape of every important variable before using it: string, number, array of rows, single row object, or undefined.
- Practice small PostgreSQL queries separately before adding them into Express routes.
- Use `req.body` and `req.params` intentionally. Do not guess; match them to form names and route parameters.
- Practice defensive checks: no matching country, duplicate country, no active user, and empty form values.

Current Project Snapshot

- The app stores an active user id in `currentId`.
- The home route fetches all users, finds the current user, and renders only the countries visited by that user.
- The add route inserts the selected country code with the active user id.
- The user route switches between existing users or opens the new-user screen.
- The new route inserts a new user and sets that new user's id as the active user.

Recommended Next Steps

- Add a duplicate-country guard so the same user cannot insert the same country twice.
- Show friendly error messages in the input placeholder or page instead of only redirecting silently.
- Handle the case where `currentUser` is missing before using `currentUser.color`.
- Remove temporary `console.log` statements once a route is confirmed working.
- Rename unclear variables after the logic works, so the code remains readable later.
- Write a small README explaining database setup, table relationships, routes, and how to run the project.

Final Note

Your strongest sign of progress is that you began explaining the logic in your own words. You still get stuck when several moving parts are active at once, especially when a value changes shape between browser form, Express route, SQL result, and EJS template. That is normal at this stage, but it is exactly the area to drill next.

The practical target is simple: before writing any line, say what the route receives, what it queries, what shape comes back, what variable is updated, and what the page renders. If you keep doing that, your Express and PostgreSQL work will become much less confusing.